

CDO-GIA: A Robust Textual Gradient Inversion Attack against Federated Language Models via Continuous-Discrete Optimization

Jiajie Wang^{1,2*}, Weibo Xu^{1*}, Ruichen Xia^{3,4†} and Jiahao Nie^{1†}

¹School of Information Technology and Artificial Intelligence,
Zhejiang University of Finance and Economics, Hangzhou 310018, China

²Raina Technology, Shaoxing 312000, China

³College of Computer Science and Technology, Zhejiang University, Hangzhou 310027, China

⁴Institute of Computing Innovation, Zhejiang University, Hangzhou 310058, China
wangjiajie@raina.tech, xuweibo5610@zufe.edu.cn, rcxia@zju.edu.cn, jhnjie@hdu.edu.cn

1 Overview

This supplementary material provides additional details to support the findings and reproducibility of the proposed CDO-GIA framework. Specifically, **Appendix 2** elaborates on the Tabu Beam Search (TBS) algorithm, presenting the complete pseudocode and a detailed workflow description of the discrete optimization phase. Subsequently, **Appendix 3** offers comprehensive implementation details, including the statistical characteristics of the datasets, hardware and software specifications, target model configurations, and a complete list of hyper-parameters for both CDO-GIA and baseline methods.

2 Details of Tabu Beam Search

The discrete sequence obtained from the Embed2Seq mapper often contains position errors or local inconsistencies. To address this, we introduce the Tabu Beam Search (TBS) mechanism. The complete algorithmic procedure is detailed in Algorithm 1.

Workflow Description. The process begins by initializing the best solution $(\mathbf{x}_{\text{best}}, \mathbf{t}_{\text{best}})$ with the output of the Embed2Seq mapper. A priority queue Q is established to maintain the beam of the top- B candidate sequences, ordered by their heuristic values $h(\mathbf{x}, \mathbf{t})$. To prevent cyclic searching and revisiting suboptimal states, a Tabu list $\mathcal{T}$ is maintained with a maximum capacity L .

The search proceeds iteratively for K rounds. In each iteration, the algorithm retrieves all current candidates from Q . For every candidate, four distinct discrete transformation operators (T_1 to T_4) are applied independently to generate new potential sequences. Each newly generated candidate is then checked against the Tabu list. If it has not been visited, we evaluate it using the heuristic function. If a new candidate surpasses the current global best solution, $\mathbf{x}_{\text{best}}$ and $\mathbf{t}_{\text{best}}$ are updated immediately. Valid candidates are then added to the Tabu list and pushed into the priority queue. Finally, at the end of each iteration, the beam size constraint is enforced by retaining only the top- B candidates in Q .

*These authors contributed equally to this work.

†Corresponding author.

Algorithm 1 Tabu Beam Search (TBS)

```
1: Input: Initial embedding  $\mathbf{x}$ , sequence  $\mathbf{t}$ , iterations  $K$ ,  
   beam size  $B$ , tabu limit  $L$ .  
2: Output: Best candidate  $(\mathbf{x}_{\text{best}}, \mathbf{t}_{\text{best}})$ .  
3: Initialize:  
4:  $\mathbf{x}_{\text{best}} \leftarrow \mathbf{x}, \mathbf{t}_{\text{best}} \leftarrow \mathbf{t}$   
5:  $Q \leftarrow \text{PriorityQueue}(\text{Capacity}=B); \mathcal{T} \leftarrow \emptyset$   
6:  $Q.\text{Push}(\mathbf{x}, \mathbf{t}, h(\mathbf{x}, \mathbf{t}))$   
7: for  $k = 1$  to  $K$  do  
8:    $\mathcal{C}_{\text{curr}} \leftarrow Q.\text{PopAll}()$  {Retrieve all candidates in current  
   beam}  
9:    $Q.\text{Clear}()$   
10:  for all  $(\mathbf{x}_c, \mathbf{t}_c) \in \mathcal{C}_{\text{curr}}$  do  
11:    for  $j = 1$  to  $4$  do  
12:       $(\mathbf{x}', \mathbf{t}') \leftarrow \text{Apply } T_j \text{ to } (\mathbf{x}_c, \mathbf{t}_c)$  {Generate 4 vari-  
      ants per candidate}  
13:      if  $(\mathbf{x}', \mathbf{t}') \notin \mathcal{T}$  then  
14:         $\text{score} \leftarrow h(\mathbf{x}', \mathbf{t}')$   
15:        {Update global best solution}  
16:        if  $\text{score} < h(\mathbf{x}_{\text{best}}, \mathbf{t}_{\text{best}})$  then  
17:           $(\mathbf{x}_{\text{best}}, \mathbf{t}_{\text{best}}) \leftarrow (\mathbf{x}', \mathbf{t}')$   
18:        end if  
19:         $\mathcal{T}.\text{Add}(\mathbf{x}', \mathbf{t}')$  {Update Tabu list}  
20:        if  $|\mathcal{T}| > L$  then  
21:           $\mathcal{T}.\text{RemoveOldest}()$   
22:        end if  
23:         $Q.\text{Push}(\mathbf{x}', \mathbf{t}', \text{score})$   
24:      end if  
25:    end for  
26:  end for  
27:  {Pruning: Keep only top- $B$  candidates}  
28:  if  $Q.\text{Size}() > B$  then  
29:     $Q.\text{KeepTop}(B)$   
30:  end if  
31: end for  
32: return  $(\mathbf{x}_{\text{best}}, \mathbf{t}_{\text{best}})$ 
```

3 Implementation Details

3.1 Datasets

To comprehensively evaluate the robustness and generalization capability of CDO-GIA, we conduct experiments on

three representative binary text classification benchmarks. These datasets are selected to cover a diverse range of domains, sequence lengths, and semantic complexities. The detailed statistics are summarized in Table 1.

Dataset	Total Samples	Avg. Len.	Std. Dev.	Vocab Size	Class 1 Ratio	Class 2 Ratio
CoLA	10,657	7.3	1.8	11,345	0.53	0.47
SST-2	67,349	9.7	2.4	16,428	0.51	0.49
RT	10,662	21.5	3.7	12,892	0.52	0.48

Table 1: Statistical characteristics of the CoLA, SST-2, and Rotten Tomatoes (RT) datasets.

- **Corpus of Linguistic Acceptability (CoLA)** [Warstadt *et al.*, 2019]: A benchmark dataset focusing on grammatical acceptability. It comprises 10,657 English sentences extracted from linguistic publications, annotated by experts as either grammatical or ungrammatical. The dataset features compact sentence structures with an average length of 7.3 words and a standard deviation of 1.8 words, typically ranging from 5 to 9 words.
- **Stanford Sentiment Treebank (SST-2)** [Socher *et al.*, 2013]: A classic benchmark for sentiment analysis containing 67,349 movie review snippets labeled with binary sentiment (positive or negative). The sequence lengths are moderately concentrated (average 9.7 words, std 2.4 words), typically ranging between 3 and 13 words. Unlike CoLA, SST-2 features rich colloquial expressions and emotional modifiers typical of short reviews.
- **RottenTomatoes (RT)** [Pang and Lee, 2005]: This dataset consists of 10,662 movie reviews collected from the Rotten Tomatoes website. It is characterized by significantly longer sequences compared to CoLA and SST-2, with an average length of 21.5 words and a standard deviation of 3.7 words (typically 14–27 words). The reviews often contain detailed descriptions of plot and acting, presenting a higher level of semantic complexity.

3.2 Experimental Environment

All experiments were conducted on a high-performance workstation. The detailed hardware specifications and software versions are listed below:

- **CPU:** AMD Ryzen 7 3700X 8-Core Processor @ 3.6 GHz
- **GPU:** NVIDIA GeForce RTX 3090 (24 GB VRAM)
- **RAM:** 64 GB DDR4
- **OS:** Ubuntu 20.04 LTS
- **Software Framework:** Python 3.9.7, PyTorch 1.10.2, CUDA 11.3

In this section, we provide a comprehensive description of the experimental setup, including the hardware and software environment, target model configurations, and detailed hyper-parameter settings for both CDO-GIA and the baseline methods.

Method	Hyper-parameter	Value
DLG / TAG	Optimization Rounds	2,500
	Learning Rate	0.2
LAMP	Total Iterations (n_i)	8
	Continuous Steps (n_c)	250
	Init Samples (n_0)	500
	Learning Rate	0.01
CDO-GIA	Total Iterations (n_i)	4
	Continuous Steps (n_c)	500
	Init Samples (n_0)	500
	Learning Rate (λ)	0.01
	Dynamic Decay (α_{reg})	1.0 $\rightarrow$ 0.0
	LM Weight (α_{LLM})	0.05
	Beam Size (B)	30
	TBS Iterations (K)	50
	Tabu List Size (L)	100

Table 2: Detailed hyper-parameter settings.

3.3 Model Configurations

Following the protocol in [Balunovic *et al.*, 2022], we employ the pre-trained BERT-base-uncased model [Devlin *et al.*, 2019] as the target for federated learning. The model consists of 12 transformer layers, 768 hidden units, and 12 attention heads, with a total of approximately 110M parameters. For the language model prior used in CDO-GIA to compute perplexity, we utilize the GPT-2-small model (117M parameters) from the Hugging Face Transformers library.

3.4 Hyper-parameter Settings

We utilize the Adam optimizer for the continuous optimization phase across all methods. To ensure a fair comparison, the hyper-parameters for baseline methods (DLG [Zhu *et al.*, 2019], TAG [Deng *et al.*, 2021], LAMP [Balunovic *et al.*, 2022]) are set according to their original papers or the configurations reported in [Balunovic *et al.*, 2022]. The specific settings for each method are detailed in Table 2.

References

- [Balunovic *et al.*, 2022] Mislav Balunovic, Dimitar Dimitrov, Nikola Jovanović, and Martin Vechev. Lamp: Extracting text from gradients with language model priors. *Advances in Neural Information Processing Systems*, 35:7641–7654, 2022.
- [Deng *et al.*, 2021] Jieren Deng, Yijue Wan, Ji Li, Chenghong Wang, Chao Shang, Hang Liu, Sanguthevar Rajasekaran, and Caiwen Ding. Tag: Gradient attack on transformer-based language models. In *2021 Findings of the Association for Computational Linguistics, Findings of ACL: EMNLP 2021*, pages 3600–3610. Association for Computational Linguistics (ACL), 2021.
- [Devlin *et al.*, 2019] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North*

American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers), pages 4171–4186, 2019.

- [Pang and Lee, 2005] Bo Pang and Lillian Lee. Seeing stars: Exploiting class relationships for sentiment categorization with respect to rating scales. In *Proceedings of the 43rd Annual Meeting of the Association for Computational Linguistics (ACL'05)*, pages 115–124, 2005.
- [Socher *et al.*, 2013] Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D Manning, Andrew Y Ng, and Christopher Potts. Recursive deep models for semantic compositionality over a sentiment treebank. In *Proceedings of the 2013 conference on empirical methods in natural language processing*, pages 1631–1642, 2013.
- [Warstadt *et al.*, 2019] Alex Warstadt, Amanpreet Singh, and Samuel R Bowman. Neural network acceptability judgments. *Transactions of the Association for Computational Linguistics*, 7:625–641, 2019.
- [Zhu *et al.*, 2019] Ligeng Zhu, Zhijian Liu, and Song Han. Deep leakage from gradients. *Advances in neural information processing systems*, 32, 2019.